Actividad Autonoma S8

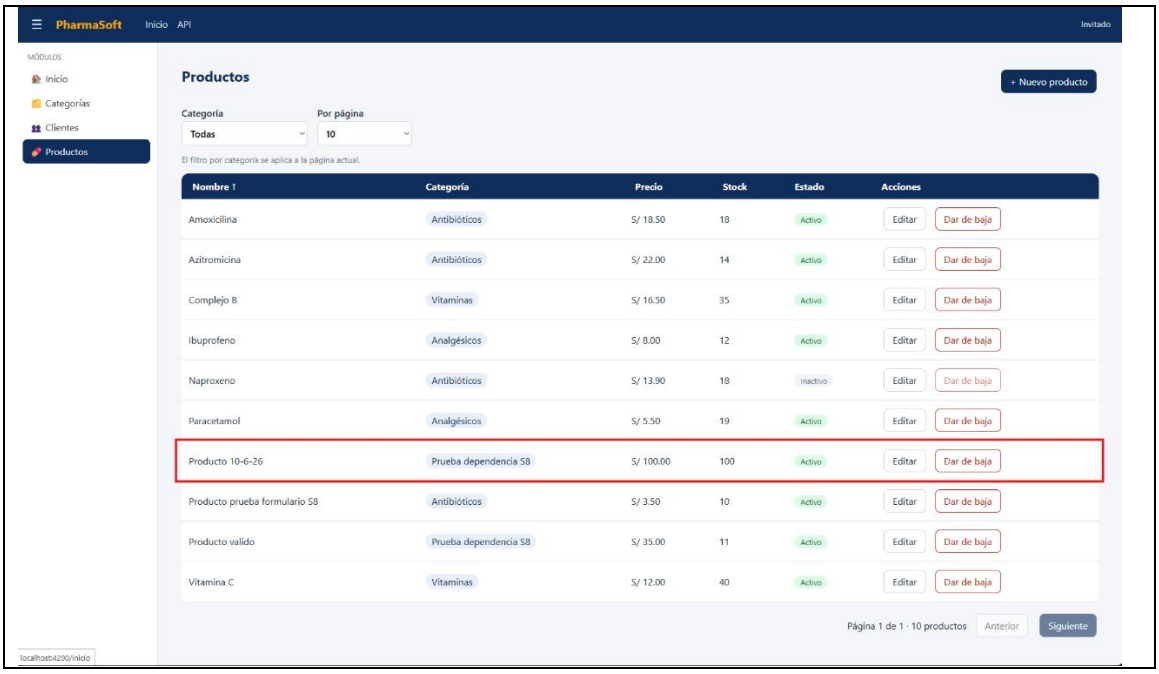
Nombre: Caleb David Asmat Briceño

Links de los repositorios:

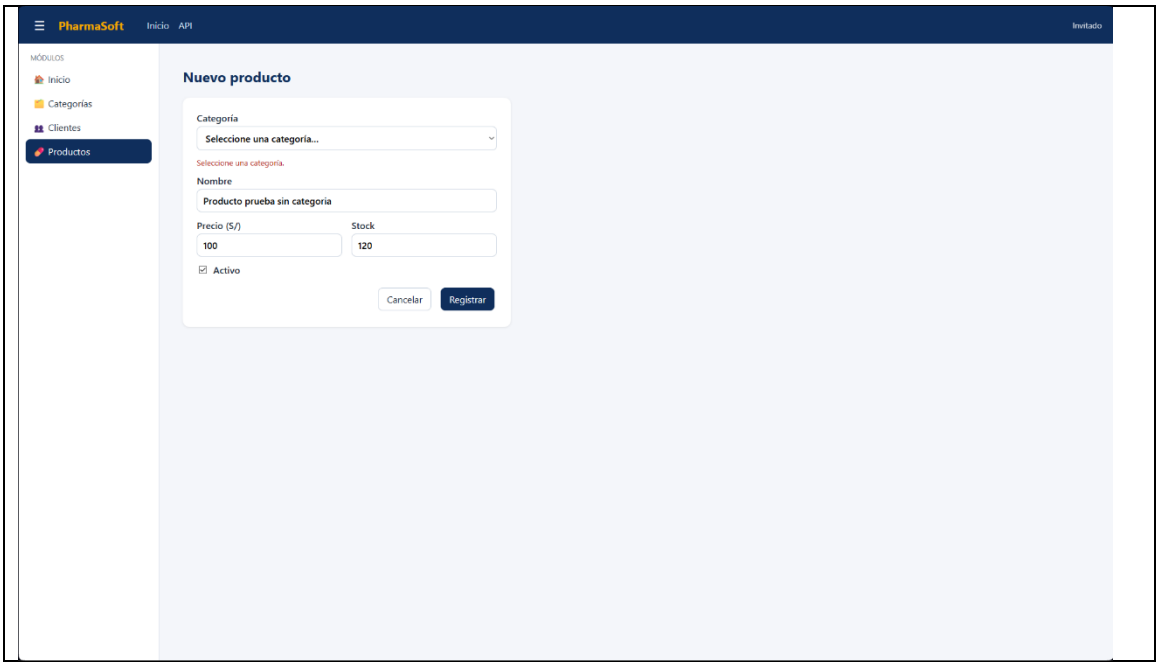
- Backend: https://github.com/ADavid2507/PharmaDB.git
- Frontend: https://github.com/ADavid2507/pharma-frontend.git

Evidencias:

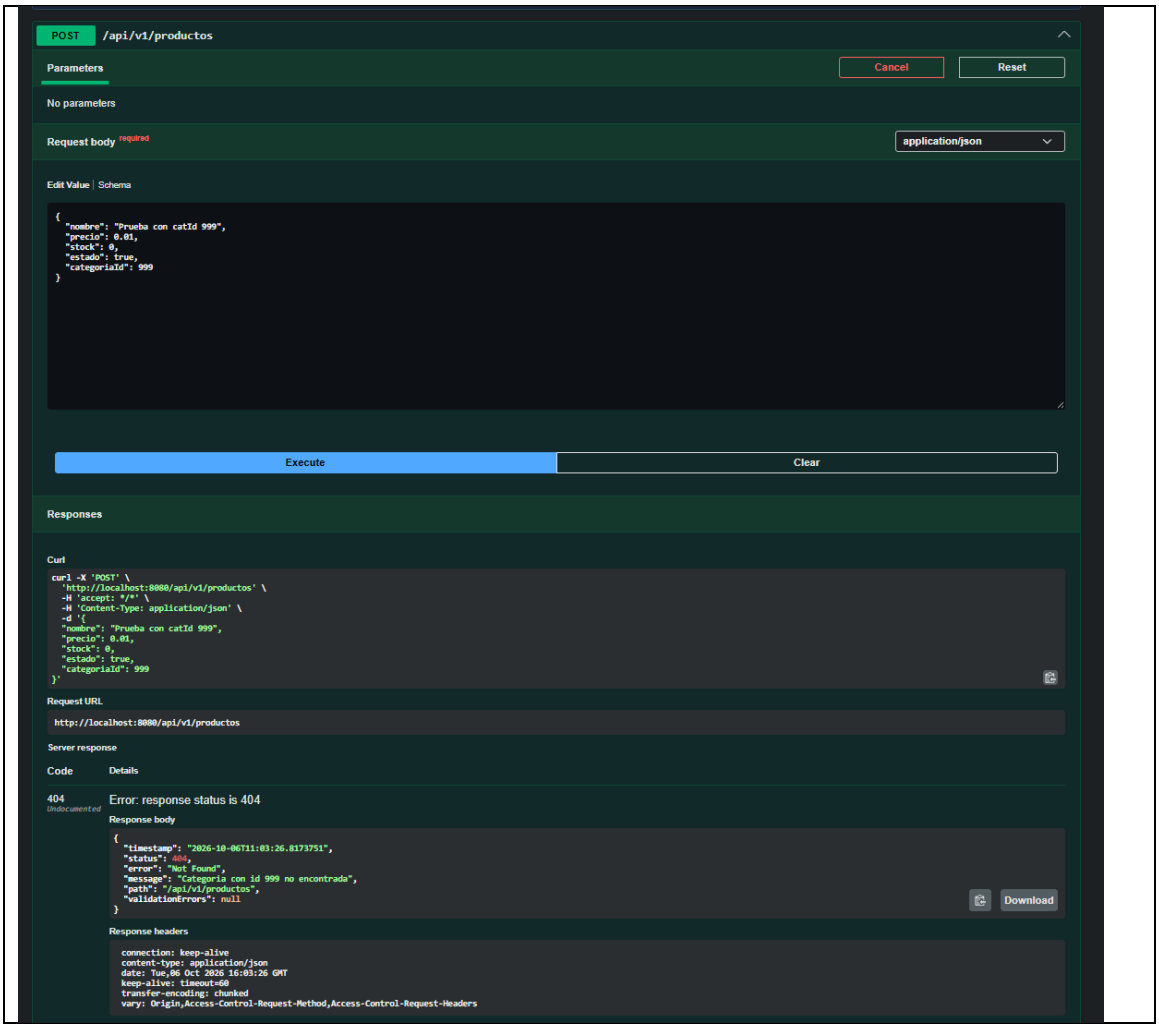
A0-1 Registrar producto valido en una categoría activa



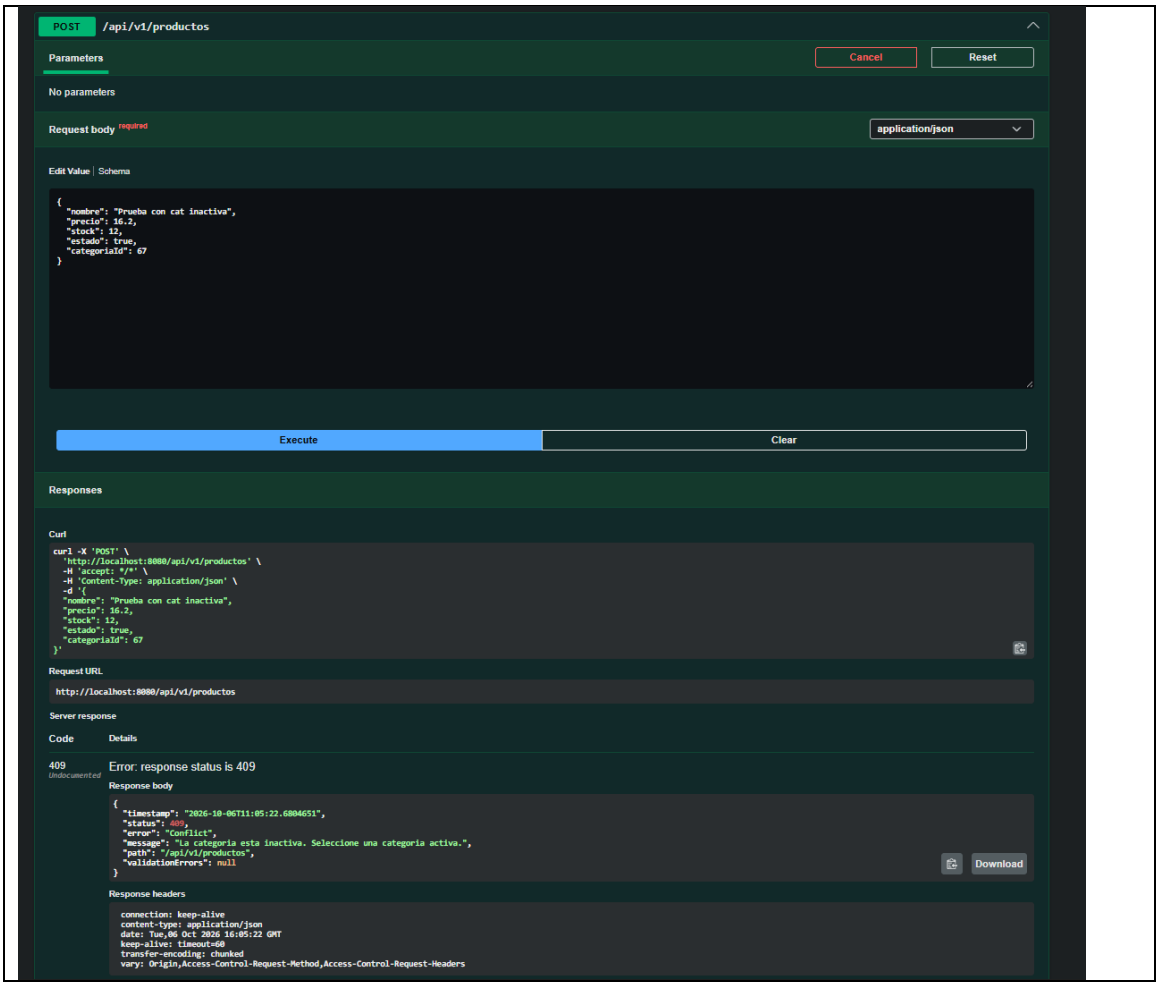
A0-2 Registrar producto sin categoría



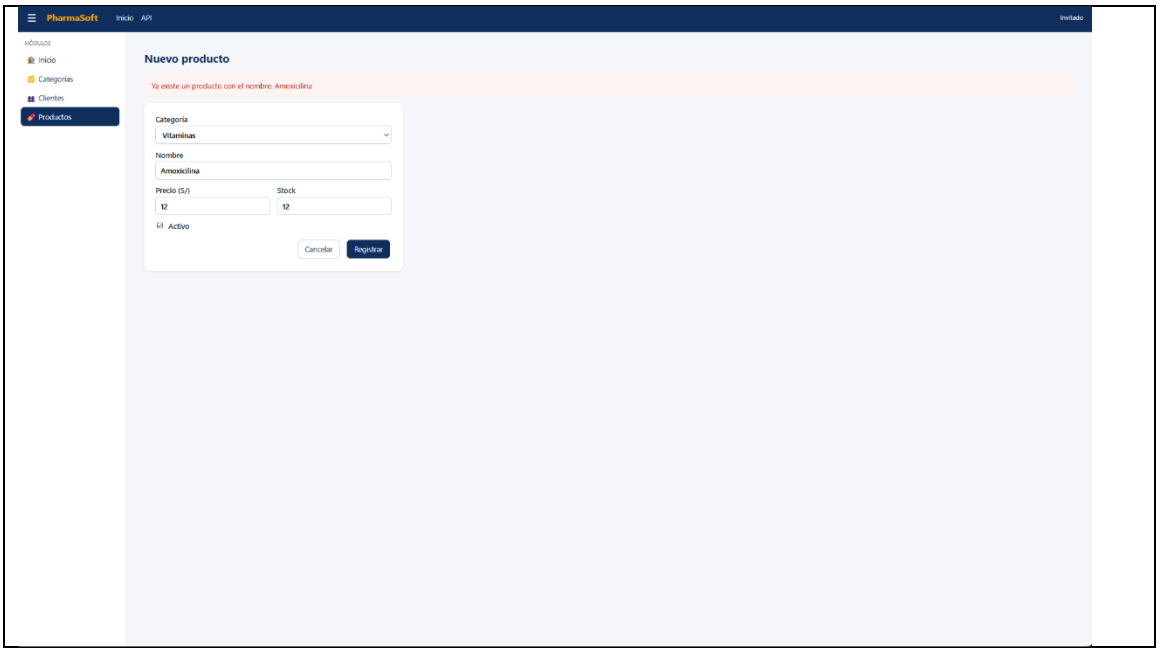
A0-3 Registrar producto con categoría – 999



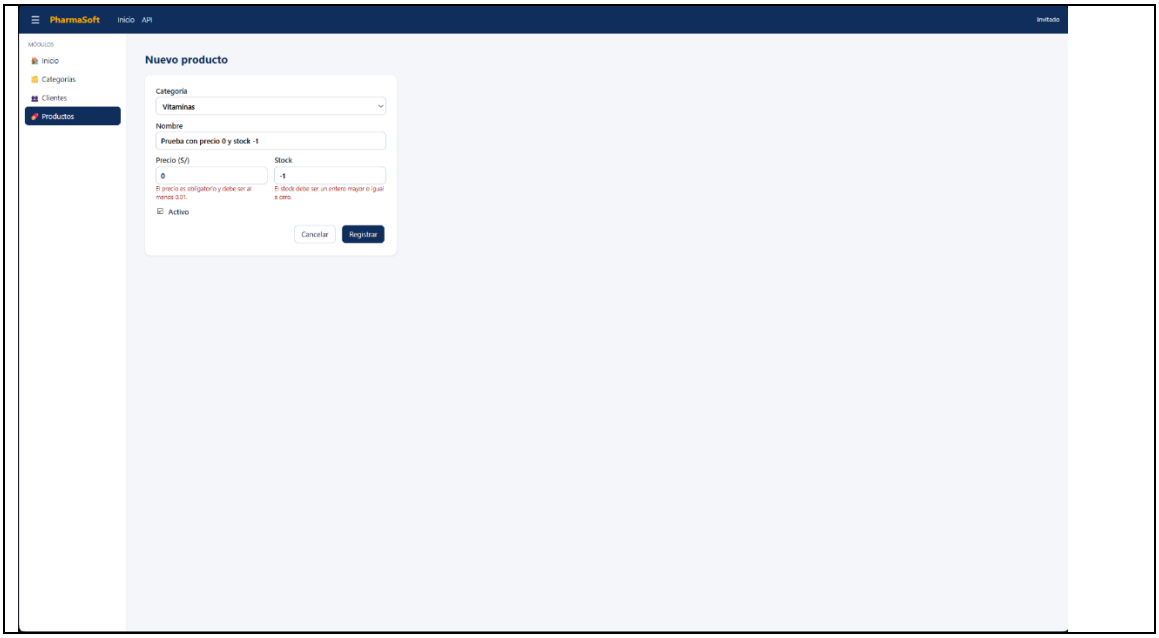
A0-4 Registrar producto en una categoría inactiva



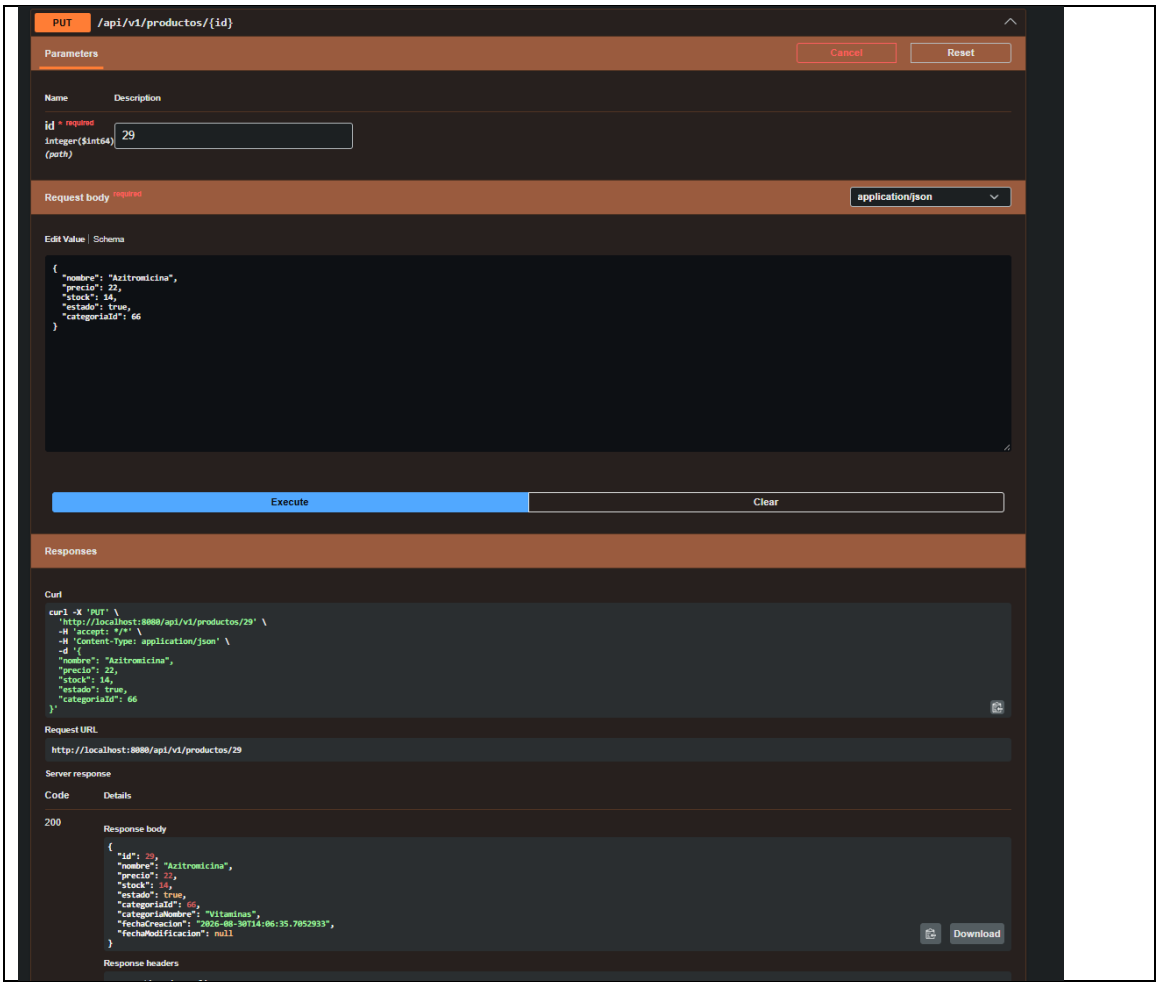
A0-5 Registrar producto con nombre existente



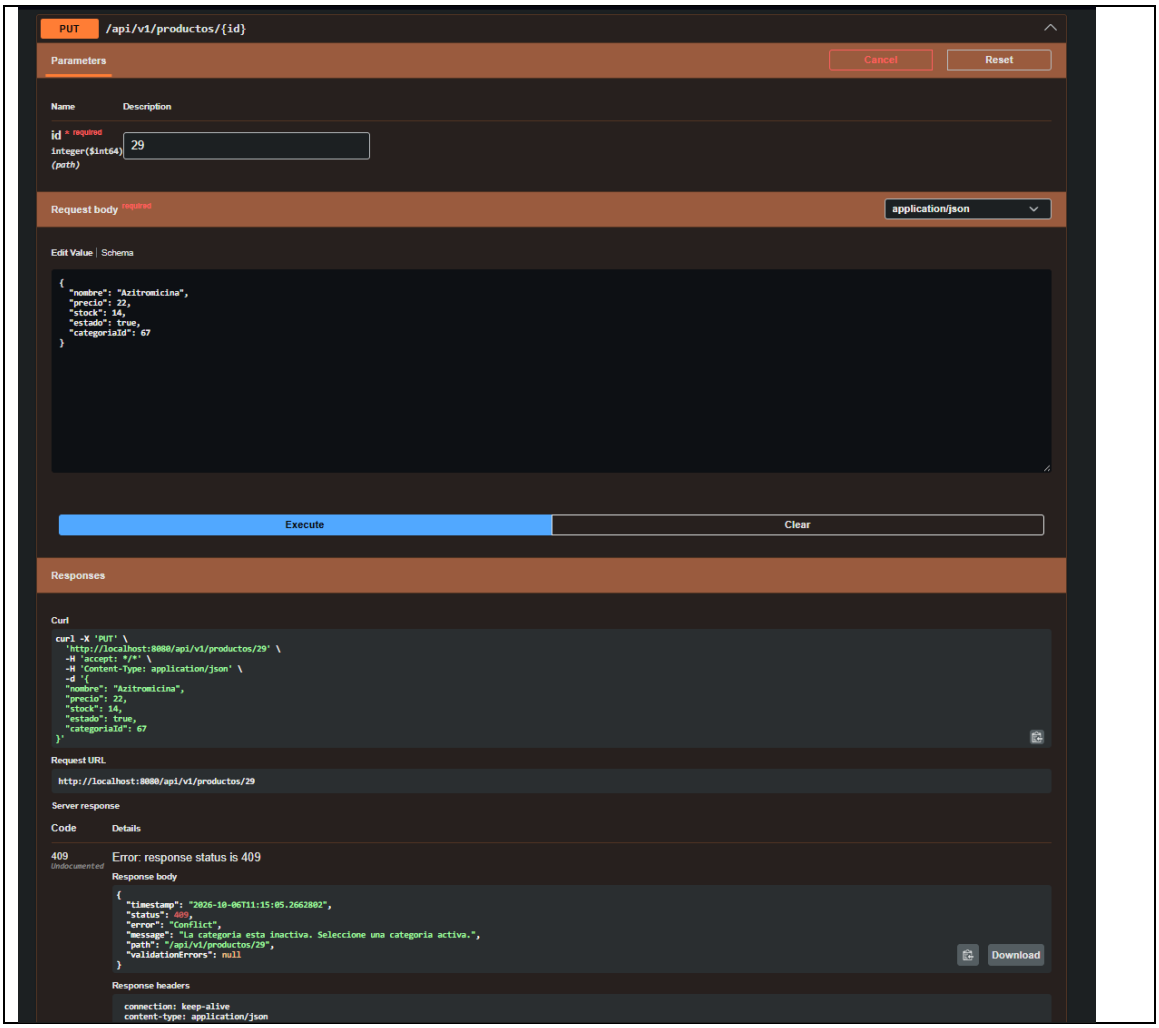
A0-6 Registrar producto con precio 0, y stock -1



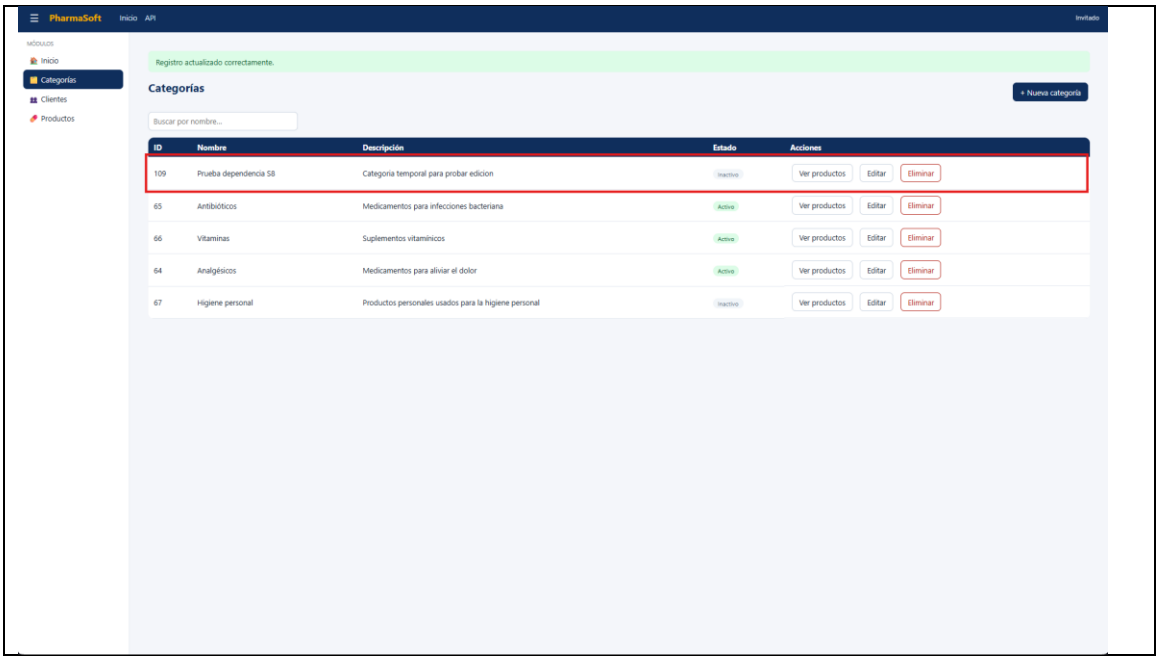
C-01 Cambiar la categoría de un producto a otra activa.



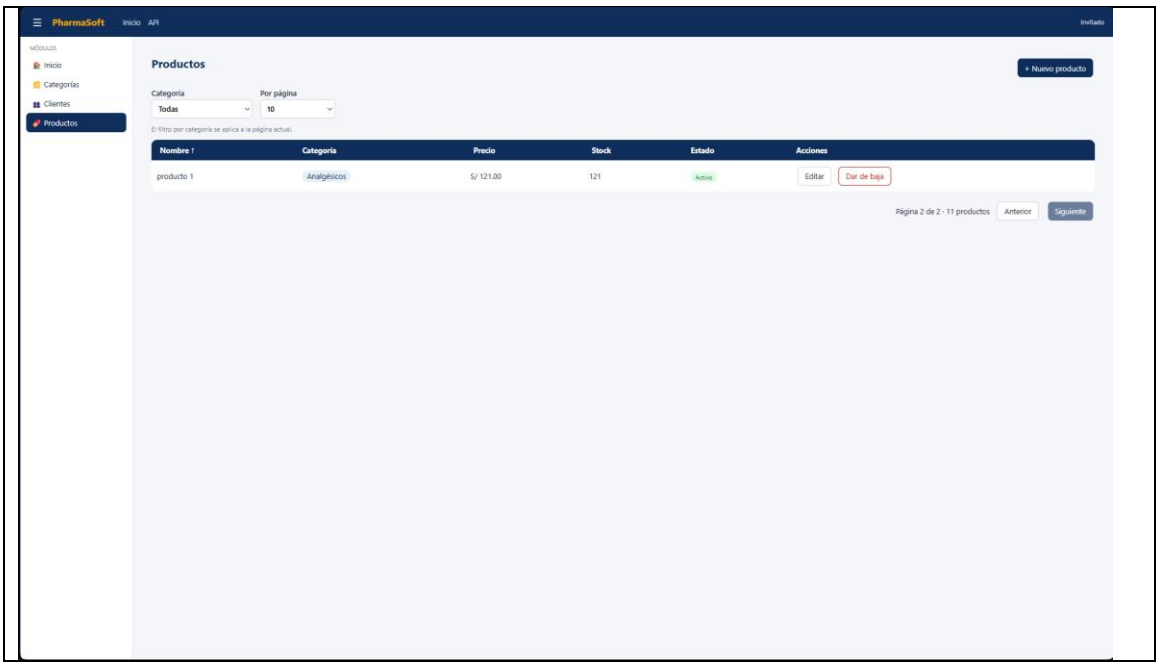
C-02 Cambiar la categoría de un producto a una inactiva.

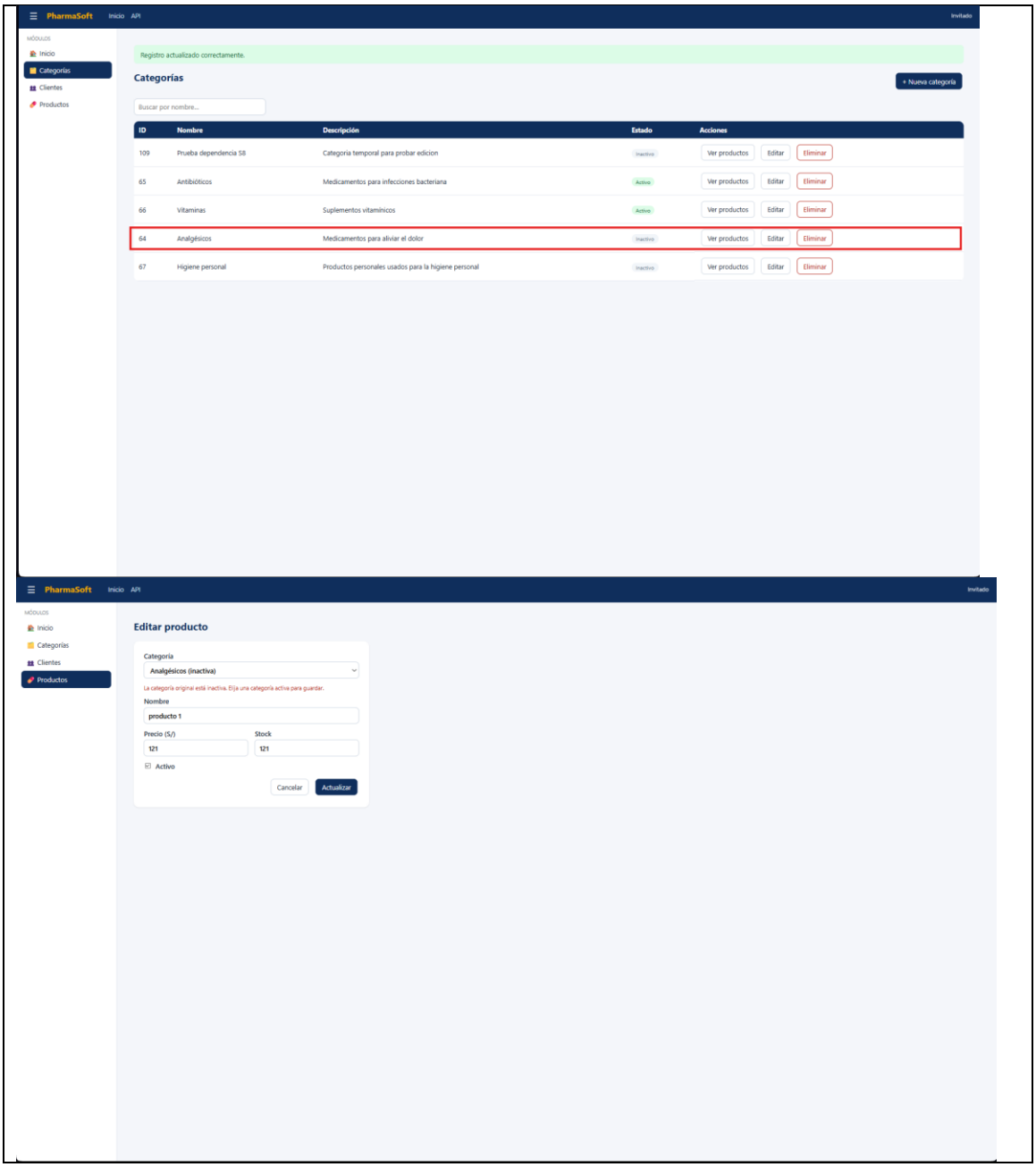


C-03 Desactivar una categoría que tiene productos activos.

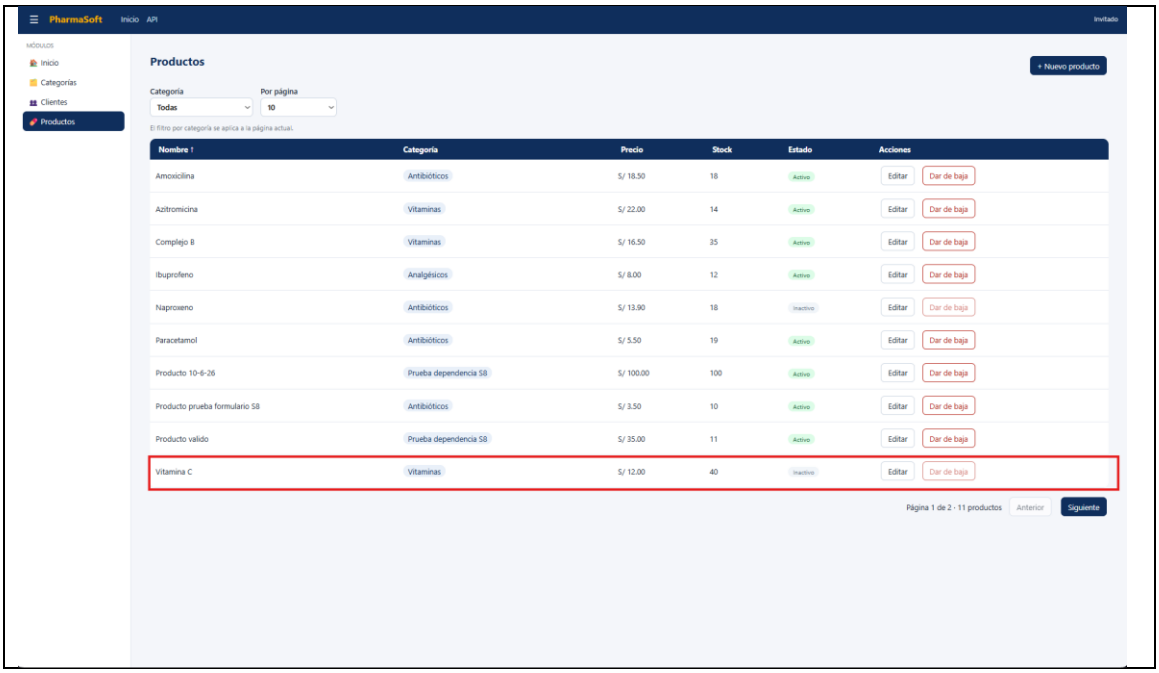


C-04 Dos pestañas: 1 abrir nuevo producto con la categoría x en la 2 desactiva x volver a la 1 y registrar.

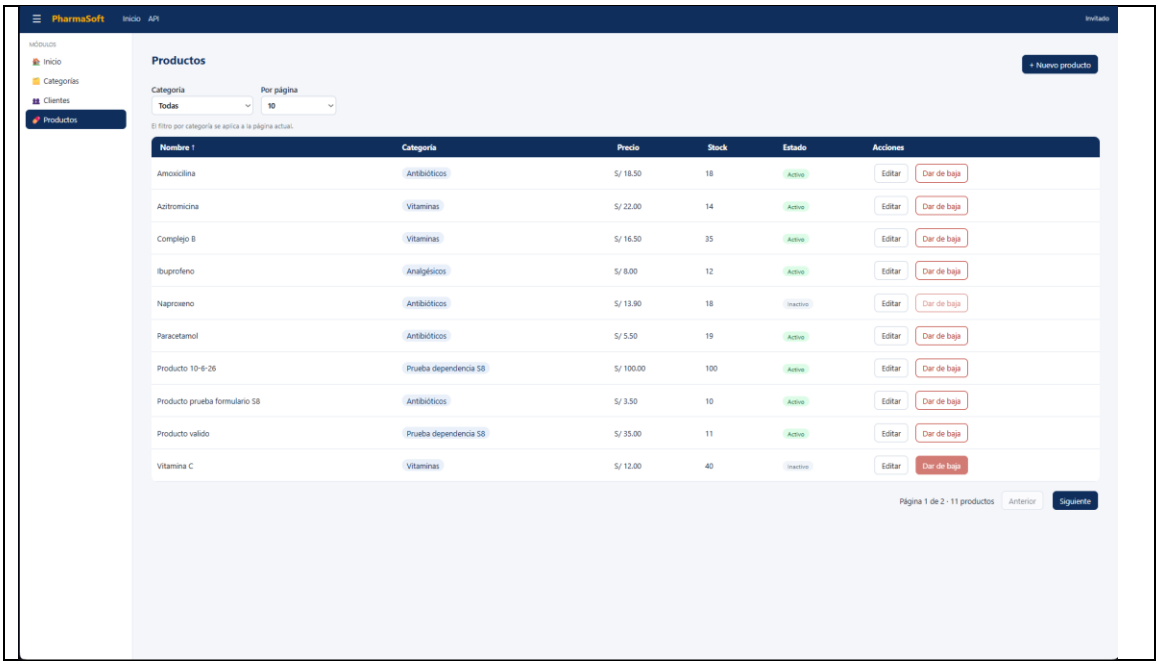




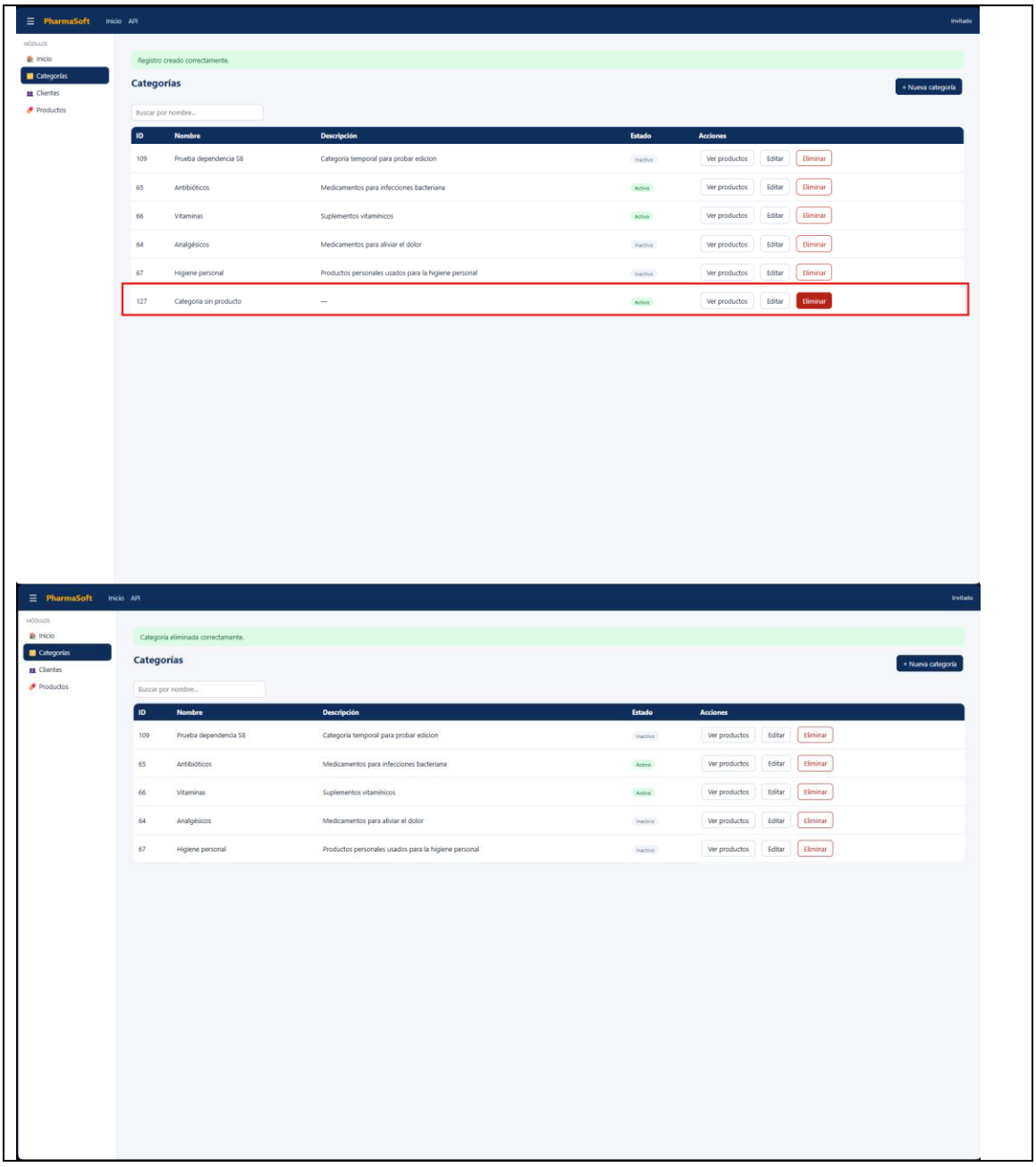
B-01 Dar de baja un producto activo



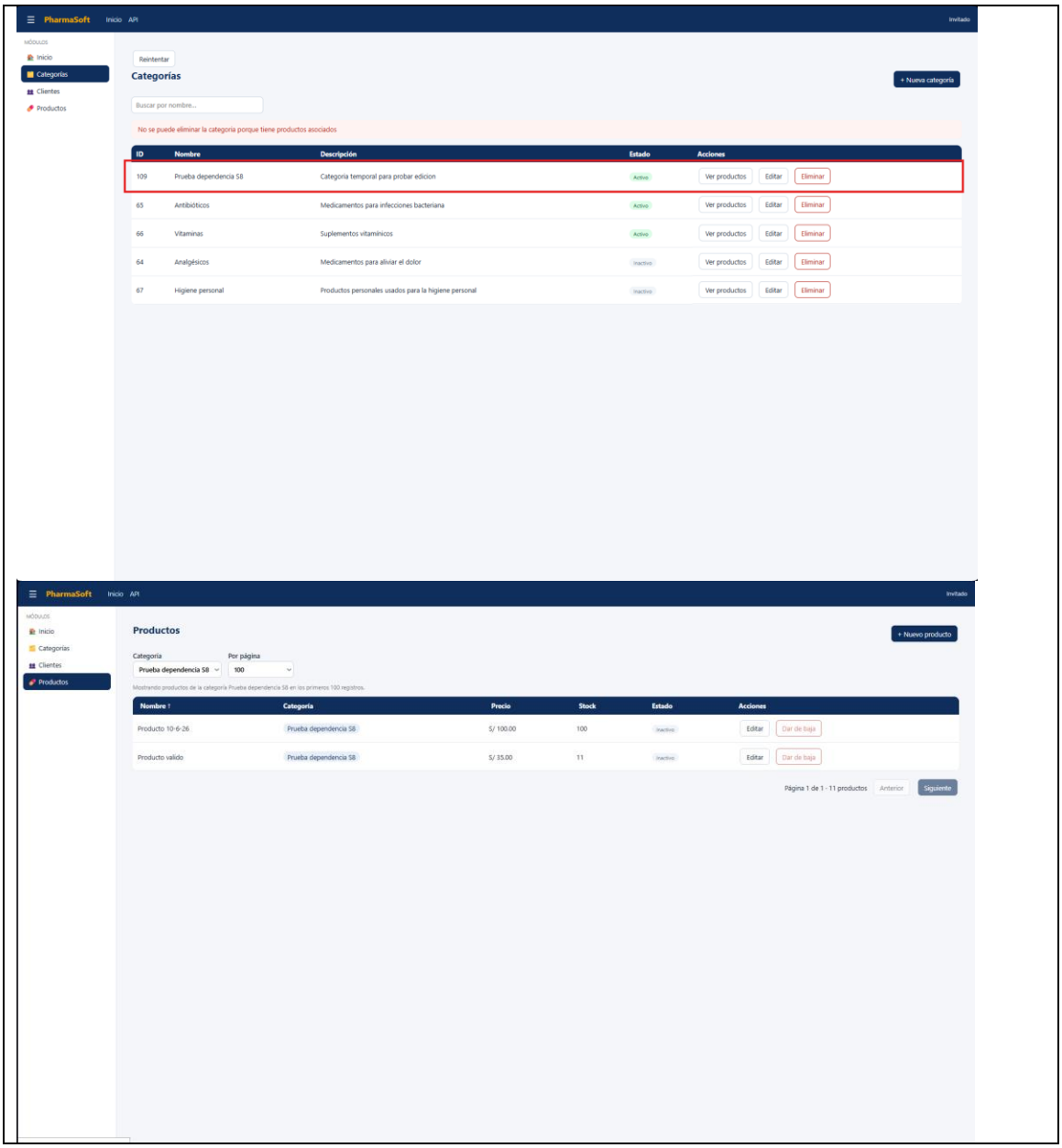
B-02 Dar de baja al mismo producto.



B-03 Eliminar una categoría sin productos.

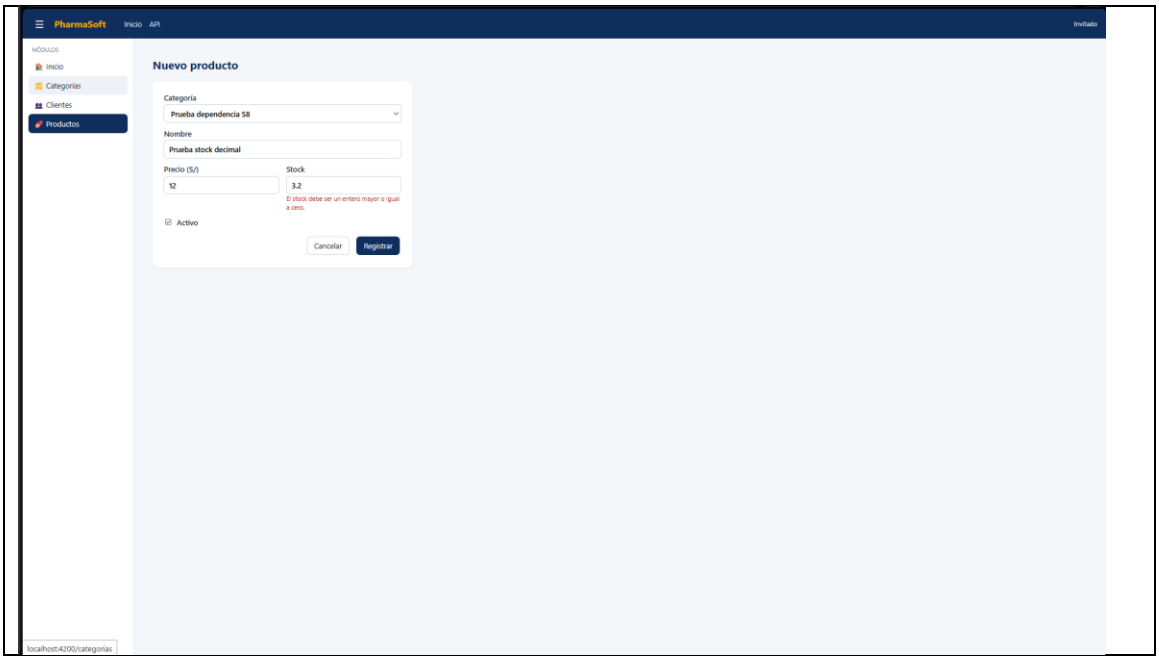


B-04 Eliminar categoría con todos sus productos inactivos

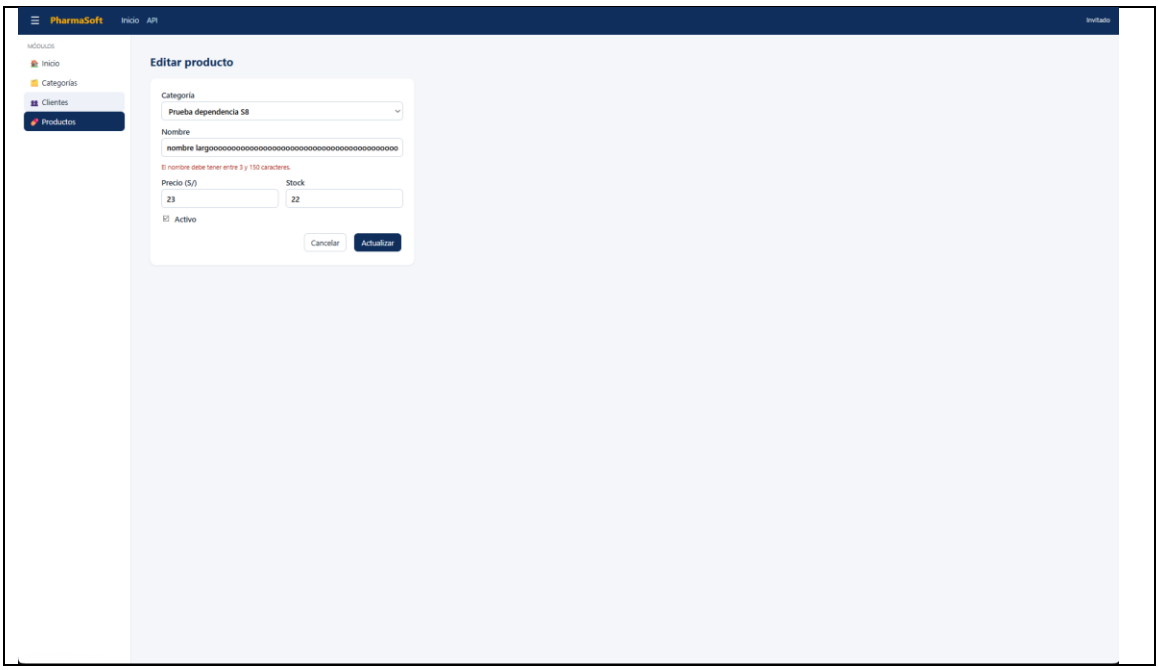


Pruebas adicionales:

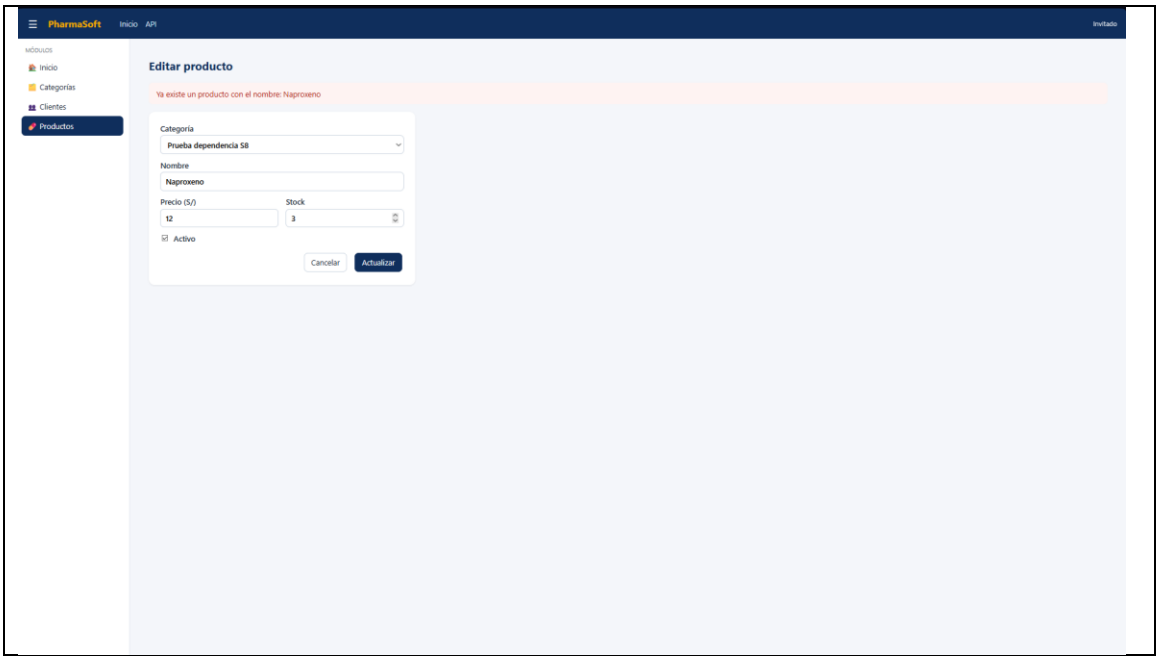
A-07 Registrar con stock decimal



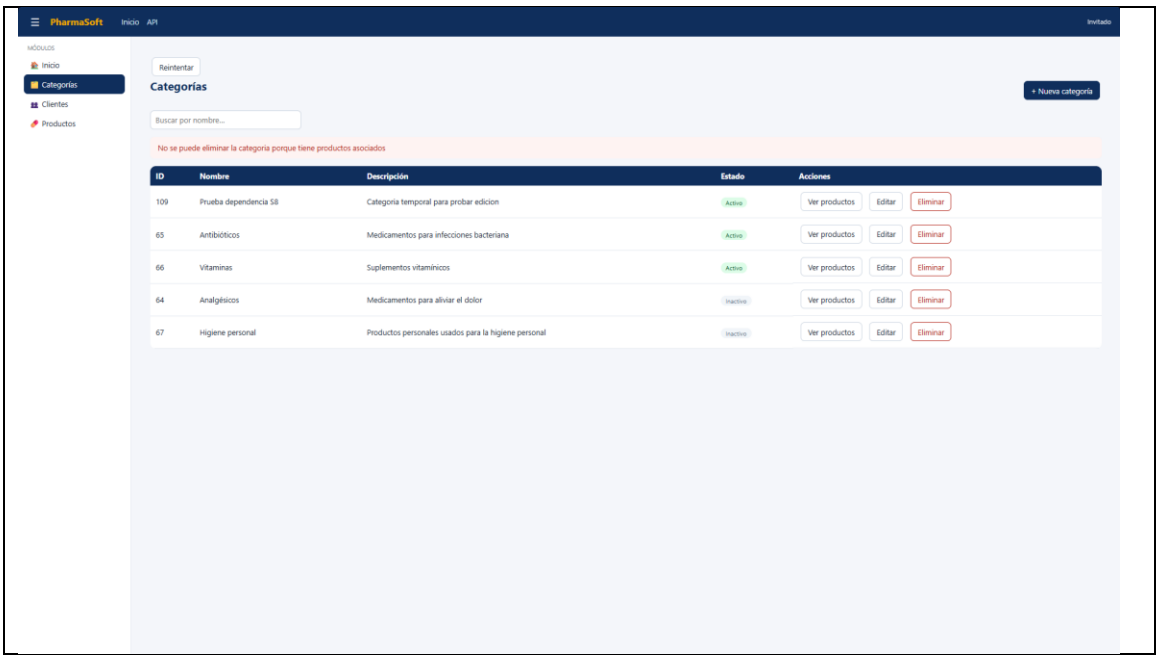
A-08 Registrar con nombre demasiado largo



C-05 Renombrar un producto usando el nombre de otro



B-05 Eliminar una categoría con productos activos



PARTE B. INFORME DE HALLAZGOS

B1. Resumen de la ejecución

Se preparó una matriz de 18 casos: 14 establecidos por la guía y cuatro propios. Se distribuyen en ocho altas, cinco cambios y cinco bajas. La evaluación compara las restricciones de la SPA con las respuestas de la API, incluyendo solicitudes directas desde Swagger que la interfaz normalmente impide.

Las pruebas utilizaron 3 categorías activas, 2 categorías inactivas y 11 productos, de los cuales 4 estaban dados de baja. Los identificadores y datos específicos se registran en la matriz.

B2. Fichas de hallazgos

La revisión del código identifica un posible incumplimiento en C-03: la actualización de Categorías permite cambiar su estado sin comprobar si tiene productos activos. Debe confirmarse mediante la ejecución del caso.

Para C-03 se eligió esta regla: impedir la desactivación de una categoría mientras tenga productos activos.

H-01. Desactivación de categoría con productos activos

Campo	Contenido
Caso relacionado	**C-03.**
Descripción	Al desactivar una categoría con productos activos, el sistema respondió desactivando la categoría sin ninguna restriccion. Esto contradice la regla elegida, que exige responder 409 y conservar la categoría activa.
Severidad	Alta, por incumplir la política definida para los estados de categorías y productos. La relación se conserva, pero queda una categoría inactiva asociada a productos activos.
Evidencia	Captura C-03
Causa probable	El método update() de CategoriaServiceImpl.java aplica los cambios mediante el mapper y guarda la entidad sin comprobar productos activos asociados.
Corrección propuesta	Antes de desactivar, comprobar si existen productos activos. Si existen, lanzar ReglaNegocioException y devolver 409. Aplicar la misma regla en cualquier otro endpoint que permita desactivar categorías.
Estado	Propuesta, no implementada. [

La consulta propuesta para ProductoRepository es:
boolean existsByCategoriaIdCategoriaAndEstadoTrue(Long categoriaId);
La validación pertenece al servicio del backend, porque debe proteger tanto las solicitudes de la SPA como las enviadas directamente a la API. La interfaz debe mostrar el mensaje y permitir revisar las dependencias mediante «Ver productos».

B3. Cobertura de las reglas

Regla	**Dónde se valida**	**Evidencia en el código**
Existencia de la categoría	Ambas, con alcances distintos	ProductoForm.guardar() comprueba la selección contra las categorías cargadas. ProductoServiceImpl.create() y update() consultan CategoriaRepository.findById(). La API verifica los datos actuales.
Categoría activa al registrar o actualizar	Ambas	El formulario filtra opciones y valida la selección. Los métodos create() y update() del backend comprueban el estado de la categoría.
Nombre único sin distinguir mayúsculas	Solo API	ProductoServiceImpl utiliza existsByNombreIgnoreCase() y existsByNombreIgnoreCaseAndIdProductoNot(). La SPA presenta el error recibido.
No eliminar categorías con productos	Solo API	CategoriaServiceImpl.delete() utiliza existsByCategoriaIdCategoria() y rechaza la eliminación si existe cualquier producto asociado.
Baja lógica de Productos	API para persistencia; SPA para representación	El backend cambia estado a false. ProductoList recarga, conserva la fila como Inactivo y deshabilita el botón.
No desactivar categorías con productos activos	Ninguna en el flujo revisado	CategoriaServiceImpl.update() no comprueba esa dependencia. Es la regla elegida para evaluar C-03.

Referencias del frontend:
src/app/features/productos/pages/producto-form/producto-form.ts
src/app/features/productos/pages/producto-list/producto-list.ts

Referencias del backend:
Los servicios y repositorios están en
src/main/java/pe/edu/upeu/PharmaBackend/service/impl/ y
src/main/java/pe/edu/upeu/PharmaBackend/repository/.

B4. Recomendación sobre el filtro por categoría
El filtro actual trabaja únicamente sobre el arreglo resultado contenido de la página recibida. Por ello, un producto puede pertenecer a la categoría seleccionada y no aparecer si se encuentra en otra página.
La mejora «Ver productos» solicita inicialmente 100 registros y preselecciona la categoría. Esto amplía el conjunto visible, pero no implementa una búsqueda global. Para resolver esa limitación propongo:

Capa	Cambio propuesto
Endpoint	Agregar categoriaId como parámetro opcional a GET /api/v1/productos, junto con los parámetros de paginación y orden.
Repositorio	Agregar Page<Producto> findByCategoriaIdCategoria(Long categoriaId, Pageable pageable).
Servicio del backend	Usar la consulta filtrada cuando se reciba una categoría y findAll(pageable) cuando no se reciba. Los totales deben corresponder al resultado filtrado.
Servicio del frontend	Agregar categoriaId?: number a ProductoService.listar() y enviarlo mediante HttpParams cuando tenga valor.
Componente	Volver a la página 0 al cambiar de categoría, consultar la API y mostrar directamente resultado.contenido. Retirar el filtro local y la nota de los primeros 100 registros cuando se implemente el contrato global.

Ejemplo de solicitud propuesta:
GET
/api/v1/productos?pagina=0&tamanio=10&ordenarPor=nombre&direccion=asc&categoriaId=64

En el servicio del frontend, la incorporación del parámetro sería:
params = params.set('categoriaId', categoriaId);

Debe ejecutarse únicamente cuando categoriaId tenga valor. La variable params debe declararse con let, ya que HttpParams es inmutable.
Esta propuesta no está implementada en la actividad: la parte C conserva el filtro local solicitado por la guía.

B5. Preguntas de análisis

1. ¿Por qué no basta con que la SPA oculte las categorías inactivas?

Ocultar categorías inactivas evita selecciones incorrectas en la interfaz, pero no impide enviar solicitudes desde Swagger ni que una categoría cambie después de cargar el formulario. ProductoForm valida la selección, mientras ProductoServiceImpl.create() y update() consultan la categoría y comprueban su estado antes de guardar. La protección no puede depender exclusivamente de los datos del navegador.

2. ¿Qué debería ocurrir al desactivar una categoría con productos activos?

Elijo impedir la desactivación y responder 409 mientras existan productos activos asociados. El usuario debe reasignarlos a otra categoría activa o darles de baja según corresponda. Esto evita desactivar productos vendibles automáticamente y mantener productos activos bajo una categoría inactiva. La regla debe implementarse en CategoriaServiceImpl.update(), utilizando una consulta de ProductoRepository.

3. ¿Es correcto impedir eliminar una categoría si sus productos están dados de baja?

Sí. La baja lógica conserva los productos y sus relaciones; no equivale a eliminarlos. CategoriaServiceImpl.delete() utiliza existsByCategoriaIdCategoria(), que considera productos de cualquier estado. Conservar la categoría mantiene la clasificación y sus relaciones históricas. Eliminarla exigiría romper, reasignar o borrar esas relaciones, lo que podría afectar la interpretación de reportes de ventas basados en ellas.

4. ¿Qué revela C-04 sobre el momento de validar las dependencias?

C-04 demuestra que los datos de un formulario pueden quedar desactualizados. Una categoría activa al abrir la primera pestaña puede estar inactiva cuando se envía el registro. La API debe volver a comprobar la dependencia durante el guardado. ProductoServiceImpl.create() realiza esa consulta y validación.\. El backend, apoyado por las transacciones y la base de datos, puede imponer la regla para todos los clientes. Las operaciones estrictamente simultáneas requieren revisar además la coordinación transaccional; esta prueba secuencial no demuestra que todas las condiciones de carrera estén resueltas.

1. **Si el backend corrigiera A-04, ¿qué cambiaría en la SPA?**

En la versión revisada, ProductoServiceImpl.create() ya rechaza categorías inactivas. Si esa validación se añadiera a una versión anterior manteniendo el formato del error, la SPA necesitaría pocos cambios: ProductoForm.guardar() ya muestra mensajeError(err) y restablece el estado de guardado. Se conservarían el filtro de categorías activas y las validaciones del formulario. Deberían actualizarse las pruebas y la matriz para verificar el 409. Solo sería necesario adaptar el manejo de errores si cambiara el formato de la respuesta.